PROJECT REPORT

AI Customer Complaint & Case Processing System

A GenAI-powered batch document processing workflow built with Python, OpenAI, LangChain and LangGraph

---

Submitted by

Ashish Jain

Enrolment / Roll Number: _____

Institution / Programme: _____

Certification Programme in Generative and Agentic AI Development

Final Project 1 of 3

Date: _____

The fields shown in red are to be completed before submission.

Table of Contents

1. Introduction

1.1 Background

Every organisation that sells a product or provides a service receives complaints, and those complaints arrive as unstructured documents: an email saved as a PDF, an intake form typed into Word, a call transcript pasted into a text file. Before anything useful can be done with a complaint, a person has to read it, identify the customer and the issue, decide whether the case needs escalation, write a reply to the customer, and write a short note so that a manager can see the state of the queue.

That work is repetitive, slow, and inconsistent between agents. It is also exactly the kind of work that a Large Language Model can do reliably, provided the output is constrained to a fixed structure rather than left as free text.

1.2 Problem Statement

Given a folder of customer complaint documents in mixed formats, automatically produce for each document: (a) a validated set of structured case fields, (b) a professional response email addressed to the customer, and (c) an internal case summary for management; and consolidate the whole batch into a single report that an operations team can act on.

1.3 Objectives

- Read complaint documents from a local folder in at least two file formats.
- Extract structured information from each document using an LLM, validated against a defined schema.
- Generate a customer response email that never invents facts absent from the source document.
- Generate a concise internal case summary with a recommended next action.
- Orchestrate the three AI tasks as an explicit workflow, using parallel execution where the tasks permit it.
- Process an entire batch automatically, isolating failures so one bad document cannot stop the run.
- Produce a consolidated report suitable for a case manager.

1.4 Scope

The system is a command-line batch application. It is deliberately not a web service: the business scenario is an overnight or on-demand batch of complaint documents, not an interactive chat. The scope covers ingestion, extraction, generation, persistence and reporting. Optical character recognition, CRM integration and a user interface are identified as future enhancements in Section 18.

2. Business Scenario

The chosen scenario is an AI Customer Complaint and Case Processing System for a consumer products company. Complaint records arrive from three channels and are dropped into a shared folder:

Channel	Typical format	Example in the sample corpus
Email, saved by the agent	.pdf	complaint_001.pdf — duplicate billing charge
Web form export	.pdf	complaint_002.pdf — delayed delivery, escalated
Phone call, transcribed	.txt	complaint_003.txt — repeat warranty failure
Intake form completed in Word	.docx	complaint_004.docx — account access / data privacy
Branch record	.docx	complaint_005.docx — service quality, resolved
Web chat	.txt	enquiry_006.txt — an enquiry, not a complaint

A typical record contains the customer's name and contact details, a free-text description of the complaint, the product or service involved, any resolution already provided, escalation notes, and a mention of supporting evidence. None of it is in a fixed position, and the wording varies by channel — which is precisely why a rule-based parser is unsuitable and an LLM is appropriate.

The sample corpus deliberately includes two edge cases: a document that is an enquiry rather than a complaint (so the system must set the complaint flag to "No"), and an unsupported file type (a .csv of courier notes) that must be skipped without error.

All names, addresses, order numbers and amounts in the sample corpus are fictional. No real customer data is used anywhere in this project.

3. System Overview

3.1 What the system does

The application is started with a single command:

```
python app.py
```

It then performs the following, without any further interaction:

1. Discovers every supported document in the data/ folder.
2. For each document, extracts the text according to its file type.
3. Runs three AI tasks over that text: structured extraction, then customer email and internal summary in parallel.
4. Writes three artefacts per document into output/, in JSON and plain text.
5. Consolidates every document into output/final_report.csv and prints a summary table.

3.2 Technology stack

Layer	Technology	Why it was chosen
Language	Python 3.10+	Standard for AI work; rich document-processing libraries.
LLM	OpenAI gpt-4o-mini	Strong structured-output support at low cost (~$0.005 per document).
LLM framework	LangChain (langchain-openai, langchain-core)	Provides prompt templates and with_structured_output, which binds a Pydantic schema to the model.
Orchestration	LangGraph	Expresses the three tasks as a graph, giving genuine parallel execution of the two independent tasks.
Validation	Pydantic v2	Defines the required output shape and validates the model's reply before the application uses it.
Document reading	pypdf, python-docx	Mature, pure-Python readers for PDF and Word.
Configuration	python-dotenv	Keeps the API key out of the source code.
Testing	pytest	15 tests that need no API key.
CI	GitHub Actions	Runs the tests automatically on every push.

4. System Architecture

The application is organised as six small modules, each with a single responsibility. Figure 1 shows how they relate to each other, to the input and output folders, and to the OpenAI API.

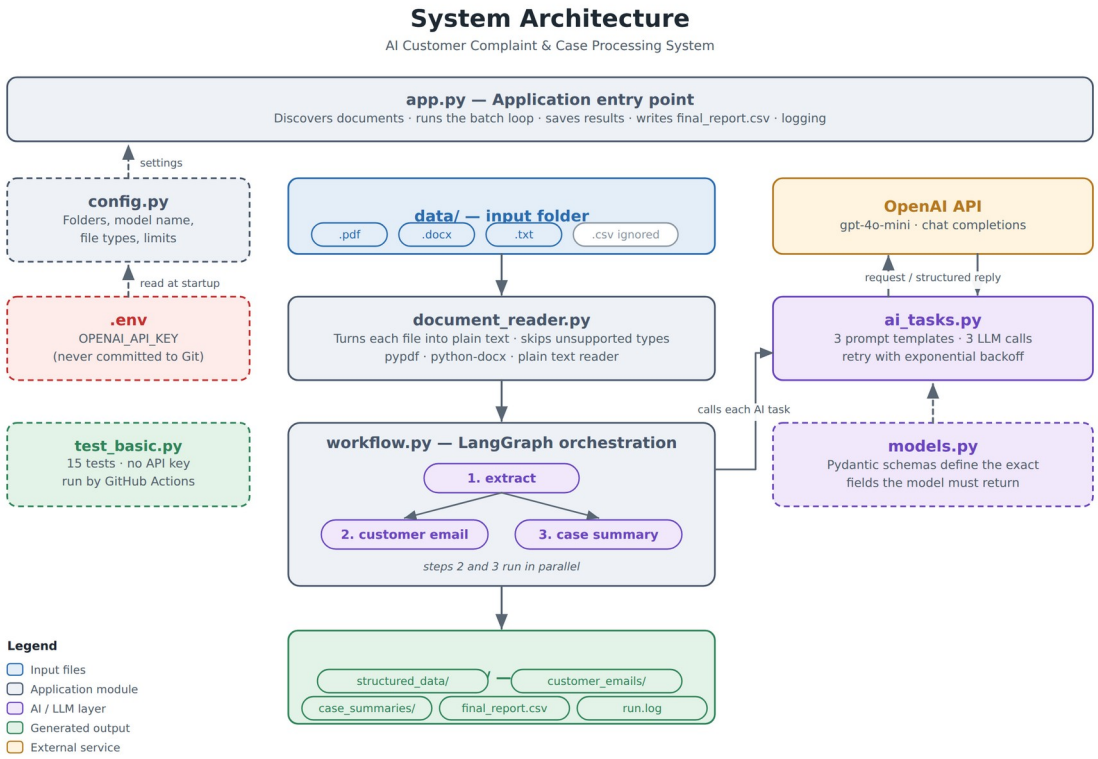

Figure 1 — System architecture

4.1 Component responsibilities

Module	Responsibility	Key contents
`app.py`	Application entry point and batch runner	main(), the per-document loop, CSV writing, logging setup
`config.py`	Single source of configuration	Folder paths, model name, temperature, supported file types, character limit
`models.py`	Defines the required output structures	ComplaintData, CustomerEmail, CaseSummary
`document_reader.py`	Turns a file into plain text	find_documents(), read_document() and one reader per format
`ai_tasks.py`	The three AI tasks	Three prompt templates, three chains, call_with_retry()
`workflow.py`	Orchestration	LangGraph State, three node functions, build_workflow()
`test_basic.py`	Automated tests	15 tests covering reading, validation and report building

4.2 Design principles

One direction of dependency. app.py depends on the other modules; none of them depends on app.py. models.py depends on nothing but Pydantic. This means any module can be imported and tested in isolation, which is what makes the test suite possible without an API key.

Separation of prompts from plumbing. Every prompt lives in ai_tasks.py and nowhere else. app.py contains no prompt text and ai_tasks.py contains no file handling, so a change to the wording of the customer email touches exactly one file.

Configuration in one place. No module reads an environment variable directly. Everything comes from config.py, so the model, the folders and the limits can be changed without searching the codebase.

The secret never enters the code. The API key is read from a .env file that is excluded by .gitignore, so it cannot be committed to GitHub by accident.

5. Workflow Design

A key requirement of the project is that the application must demonstrate workflow orchestration rather than placing everything inside one large LLM call. This system uses LangGraph to express the three AI tasks as an explicit state machine.

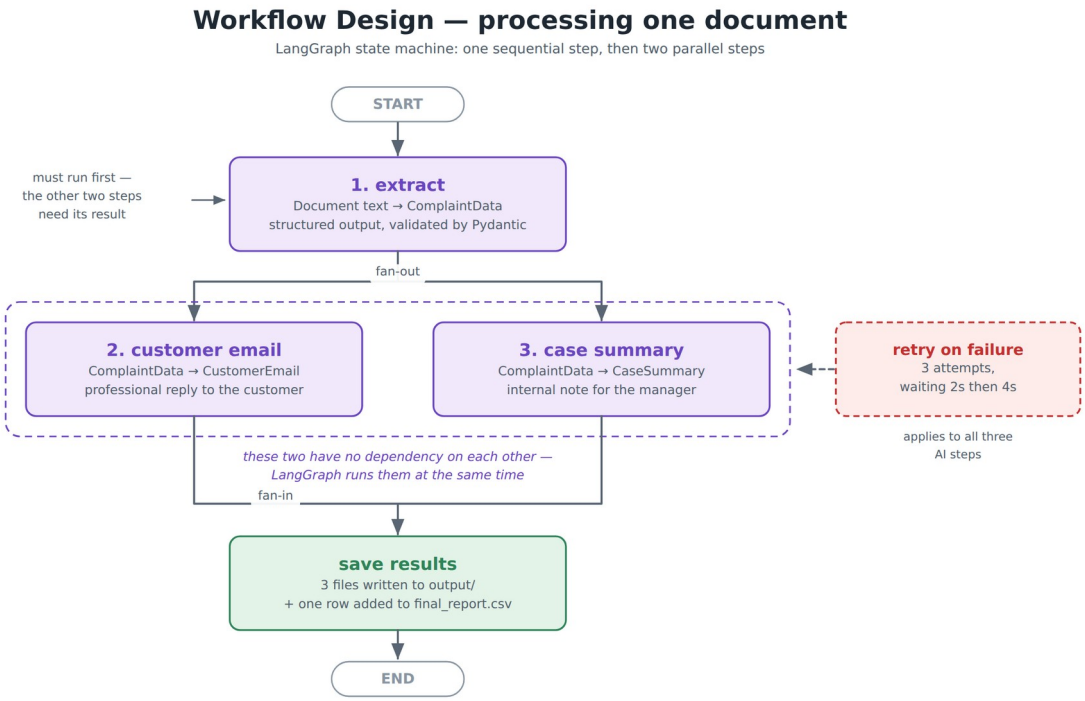

Figure 2 — Per-document workflow (LangGraph)

5.1 Sequential and parallel execution

The three tasks are not equal in their dependencies. Extraction must run first, because both of the other tasks consume its output — that is a genuine sequential dependency. The customer email and the case summary, however, depend only on the extracted data and not on each other, so there is no reason to run them one after the other.

In the graph this is expressed by drawing two edges out of the extract node. LangGraph schedules every node whose inputs are ready in the same superstep, so the email and summary calls are issued together rather than in sequence, and each document completes noticeably faster.

```
graph.add_edge(START, "extract")

# Two arrows out of one node = these steps run at the same time
graph.add_edge("extract", "email")
graph.add_edge("extract", "summary")

graph.add_edge("email", END)
graph.add_edge("summary", END)
```

The shared state is a TypedDict with four keys. Each node reads what it needs and returns only the key it produced, so the two parallel nodes write to different keys and can never conflict with each other.

```
class State(TypedDict):
```

```
    document_text: str
    complaint_data: Optional[ComplaintData]
    customer_email: Optional[CustomerEmail]
    case_summary:  Optional[CaseSummary]
```

5.2 Why a graph rather than three function calls

- Parallelism: the two independent tasks overlap, which is worthwhile because each is a network call that spends most of its time waiting.
- Clarity: the dependency between the tasks is stated once, declaratively, instead of being implied by the order of lines in a function.
- Extensibility: a fourth task — for example routing the case to a team — is added by defining one node and one edge, without touching the existing three.

5.3 Batch processing flow

Figure 3 shows the complete flow of the application from start to finish, including the two guard conditions and the error path that keeps the batch running when a single document fails.

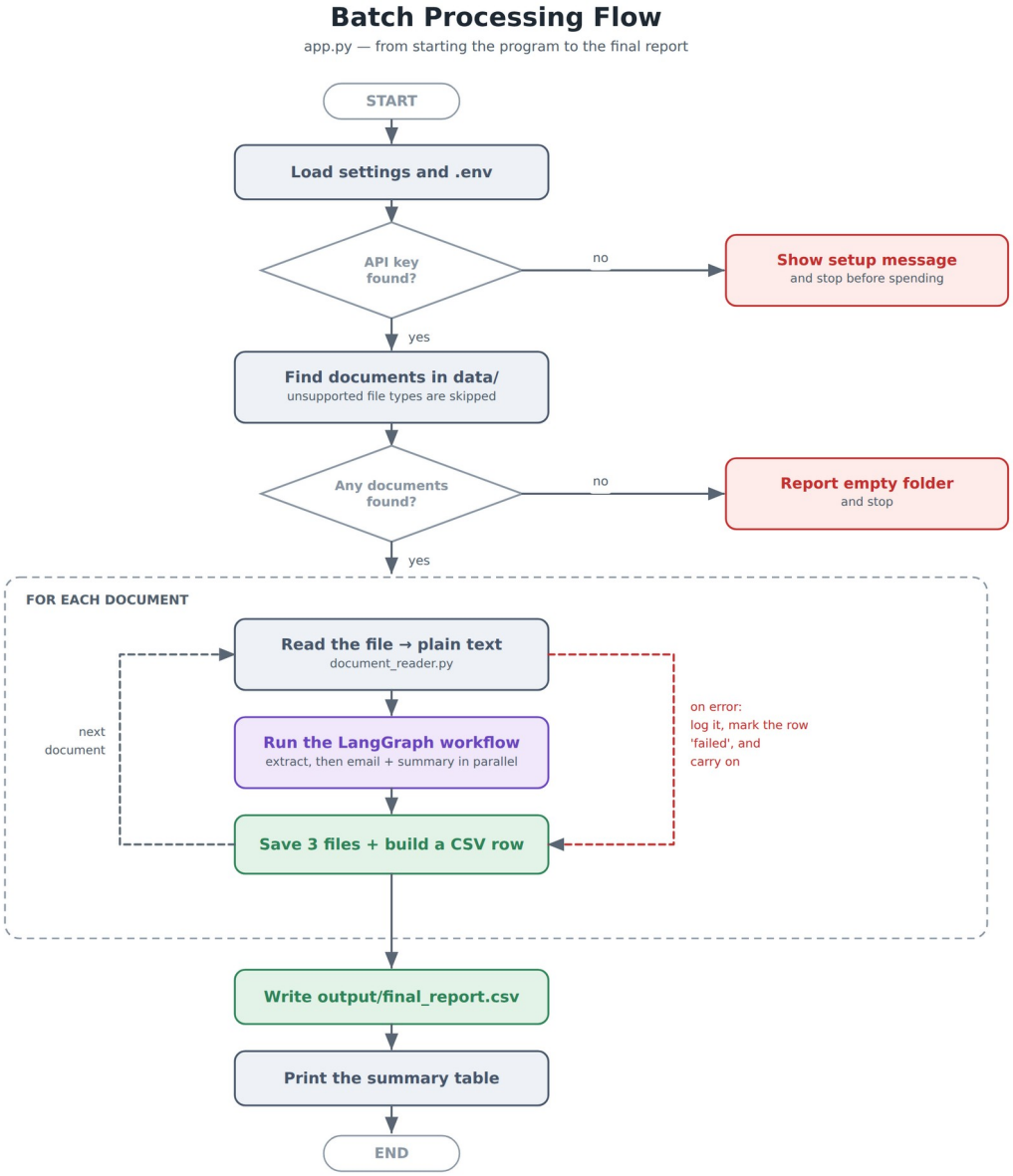

Figure 3 — Batch processing flow

6. Data Model and Structured Outputs

6.1 Why structured output matters

The application never asks the model for free text and then attempts to parse it. Instead, each task is bound to a Pydantic class using LangChain's with_structured_output. The schema is sent to the model as the required answer format; the reply comes back as JSON and is validated against the class before the application sees it.

```
chain  = EXTRACTION_PROMPT | build_llm().with_structured_output(ComplaintData)
result = chain.invoke({"document_text": text})
# result is a validated ComplaintData object, not a string
```

This has three practical consequences. The report is always machine-readable, because a category can only ever be one of the nine permitted values. A malformed reply fails immediately at the boundary rather than producing a corrupt CSV row several steps later. And the field descriptions serve double duty: they are both the documentation of the data model and part of the instruction sent to the model.

6.2 ComplaintData — the extraction schema

Field	Type	Description
customer_name	str | None	Full name; None when the document does not state it
email	str | None	Customer email address
phone_number	str | None	Customer telephone number
product_or_service	str | None	The product or service the complaint concerns
complaint_category	Literal — 9 values	Billing, Delivery, Product Defect, Service Quality, Technical Support, Account Access, Refund, Warranty, Other
issue_description	str (required)	Two to four sentences describing the problem
resolution_provided	str | None	What the company did; None when nothing is recorded
is_complaint	bool	Reported as Yes/No in the final report
escalation_required	bool	Reported as Yes/No in the final report
supporting_document_availabl e	bool	Reported as Yes/No in the final report
case_status	Literal — 6 values	Open, In Progress, Escalated, Resolved, Closed, Unknown
priority	Literal — 3 values	Low, Medium, High

Two design choices are worth noting. Optional fields default to None rather than to an empty string, which forces the model to admit when a document does not contain a piece of information instead

of inventing something plausible. And the categorical fields use Literal rather than a free string, which makes an invented category impossible: the response would fail validation.

6.3 CustomerEmail and CaseSummary

Schema	Fields	Purpose
CustomerEmail	subject, greeting, body	The reply sent to the customer. as_text() renders it as a sendable plain-text email.
CaseSummary	case_overview, key_issue, action_taken, current_status, recommended_next_action	The internal note a service manager reads before the daily review. as_text() renders it as a formatted briefing.

Both schemas expose an as_text() method so that rendering lives with the data it renders, and app.py does not need to know how an email is laid out in order to save one.

7. Prompt Engineering

7.1 Structure of the prompts

All three prompts follow the same two-message pattern. A system message fixes the role and the rules; a human message supplies only data. Keeping the rules out of the same message as the data makes the instructions harder to displace, and makes each prompt easy to review in isolation.

```
EXTRACTION_PROMPT = ChatPromptTemplate.from_messages([
    ("system", "You are a customer service analyst..." + NO_MAKING_THINGS_UP +
rules),
    ("human",  "Here is the complaint document:\n\n{document_text}"),
])
```

7.2 Preventing hallucination

Fabricated detail is the central risk in this application: an email that promises a refund the company never agreed to is worse than no email at all. Four measures are used together.

6. A shared rule, defined once and included in all three prompts: use only information present in the text provided; if something is not mentioned, leave it empty or false; never invent names, phone numbers, dates, amounts or promises.
7. Structured output, so the model fills fixed fields rather than writing free prose in which an invented detail can hide.
8. Optional fields that default to None, giving the model a legitimate way to record absence.
9. The two generation tasks receive the validated extraction rather than the raw document, so they can only work from facts that have already passed through the extraction step.

A specific instruction reinforces this in the email prompt: if no resolution is recorded, say the case is under review and give the next step — do not promise a refund or a date.

7.3 Task-specific rules

Task	Rules given to the model
Extraction	Choose one category only. is_complaint is false for an enquiry or a compliment. escalation_required is true when escalation, a manager, legal action, a regulator, or an unresolved repeated problem is mentioned. supporting_document_available is true only when evidence such as an invoice, receipt, photo or screenshot is mentioned.
Customer email	Professional, warm and clear. No marketing language, no emojis. 150–250 words. Never leave placeholders such as [insert name]. Address the customer by name, or use "Dear Customer," when no name is available.
Case summary	Write for colleagues, not for the customer. Be factual and brief. recommended_next_action must be one concrete instruction with an implied owner, for example "Billing team to confirm the refund reached the card".

The temperature is set to 0.1 in config.py. These are extraction and summarisation tasks in which consistency matters far more than creative variety.

8. Implementation Details

8.1 Document ingestion

document_reader.py exposes two functions. find_documents() lists every supported file in the input folder, sorted by name, skipping hidden files and unsupported extensions rather than raising an error. read_document() dispatches on the file suffix to one of three private readers, then applies two validations shared by all formats: a document yielding fewer than 30 characters is rejected as unreadable — which is what a scanned image-only PDF produces — and a document longer than the configured limit is truncated so that the cost of the API call stays predictable.

```
if suffix == ".txt":    text = _read_txt(path)
elif suffix == ".pdf":  text = _read_pdf(path)     # pypdf, page by page
elif suffix == ".docx": text = _read_docx(path)    # python-docx, paragraph by
paragraph
else: raise ValueError(f"Cannot read this file type: {suffix}")
```

Adding a fourth format requires one private function and one branch; no other module changes.

8.2 The AI layer

ai_tasks.py contains the three prompt templates, the three task functions, and the shared retry helper. The chat model is built once and cached with functools.lru_cache, so a single client is reused for the whole batch rather than being recreated for each of the three calls per document.

Each task function is three lines: compose the chain, invoke it through the retry helper, return the validated object.

```
def extract_complaint_data(document_text: str) -> ComplaintData:
    chain = EXTRACTION_PROMPT |
build_llm().with_structured_output(ComplaintData)
    return call_with_retry(chain, {"document_text": document_text},
"Extraction")
```

8.3 The batch runner

app.py holds the loop. Each document is wrapped in a try/except so that a failure is recorded and the loop continues. The row appended to the report contains the extracted fields with the three boolean flags converted to the Yes/No form the business requires, and, for a failed document, the reason for the failure.

```
for number, path in enumerate(documents, start=1):
    try:
        text   = document_reader.read_document(path)   # file  -> text
        result = workflow.process_document(text)       # text  -> 3 AI results
        save_results(path.stem, result)               # write 3 files
        rows.append(build_csv_row(path.name, result, time.time() - started))
    except Exception as error:
        failures += 1
        log.error("      FAILED: %s", error)
        # a 'failed' row is still added, so nothing disappears silently
```

9. Error Handling

Errors are handled at three distinct levels. The principle throughout is that a failure must be visible in the output rather than silently reducing the number of rows in the report.

Level	Where	Behaviour
API call	ai_tasks.call_with_retry()	Retries up to 3 times with exponential backoff — 2 seconds, then 4. Recovers from dropped connections, rate limits and transient server errors. After the third failure it raises a clear RuntimeError naming the task.
File	document_reader.read_document()	A missing folder raises FileNotFoundError with the path. An unsupported type, a corrupt PDF or a document with almost no text raises ValueError with an explanation the user can act on.
Document	the try/except in app.py	Any failure is logged, the document is recorded in the report with status = failed and the reason, and the batch continues with the next document.
Start-up	main()	A missing API key is detected before any document is processed, and the program exits with instructions rather than failing once per document.

This behaviour is verified by the test suite: a missing folder, an almost-empty file and an oversized document each have a dedicated test.

10. Logging

The application uses Python's standard logging module, configured once at the top of app.py. Every message is timestamped and carries a level, and each message is written both to the screen and to output/run.log, so a completed run leaves an auditable record.

```
14:02:01 | INFO    | Found 6 document(s) in data
14:02:01 | INFO    | [1/6] Processing complaint_001.pdf
14:02:05 | INFO    |       done in 3.8s
14:02:09 | WARNING | Extraction failed (attempt 1 of 3): connection reset
14:02:11 | WARNING | Waiting 2 seconds before trying again...
14:02:31 | ERROR   |       FAILED: The file contains almost no text
14:02:44 | INFO    | Final report written to output/final_report.csv
```

Levels are used deliberately: INFO for progress, WARNING for a retry that the system expects to recover from, and ERROR for a document that could not be processed.

11. Output Structure

Each run produces the following tree. The output folder is recreated on every run and is excluded from version control.

```
output/
├── structured_data/        complaint_001.json    the extracted fields
├── customer_emails/        complaint_001.txt     the reply to the customer
├── case_summaries/         complaint_001.txt     the internal note
├── final_report.csv        one row per document
└── run.log                 timestamped record of the run
```

11.1 The consolidated report

final_report.csv is the artefact an operations team would actually use. It has one row per document — including documents that failed — and eighteen columns:

Group	Columns
Identification	file_name, status
Customer	customer_name, email, phone_number
Case	product_or_service, complaint_category, issue_description, resolution_provided
Business flags (Yes/No)	is_complaint, escalation_required, supporting_document_available
Triage	case_status, priority, recommended_next_action
Reference	email_subject, seconds, error

The file is written with the utf-8-sig encoding so that Microsoft Excel opens it with the correct characters, and the three boolean fields are converted to "Yes" and "No" at the point of writing, so the business sees the vocabulary it expects while the code keeps working with real booleans.

12. Testing

The project includes fifteen automated tests in test_basic.py, run with pytest. None of them calls the OpenAI API, so the suite is free to run, needs no API key, and completes in under a second — which is what makes it suitable for continuous integration.

Area	Tests	What is verified
File processing	6	All six sample documents are found; the .csv is skipped; text, PDF and Word files are read correctly; a missing folder and an almost-empty file raise clear errors
Truncation	1	A very long document is cut to the configured limit
Pydantic models	5	Optional fields default to None; an invented category is rejected; a missing required field is rejected; the email and summary render correctly as text
Report building	2	Booleans are converted to Yes/No; the CSV columns match between the

Area	Tests	What is verified
		header and the rows

The test for an invented category is worth highlighting, because it demonstrates the guarantee that structured output provides: passing a category outside the permitted list raises a validation error rather than silently reaching the report.

13. Version Control and Continuous Integration

The project is maintained in a Git repository and published to GitHub. Two aspects are worth noting for evaluation.

Secrets are excluded. .gitignore excludes .env, output/ and the Python cache directories. The API key therefore cannot reach the repository, and generated artefacts do not pollute the commit history. A .env.example file is committed in its place so that another developer knows which variables are required.

Every push is tested. The workflow in .github/workflows/tests.yml installs the dependencies and runs pytest on every push and pull request. Because the tests need no API key, no secret has to be configured in GitHub for this to work.

The repository history is built as a sequence of small, meaningful commits — setup, schemas, document reader, AI tasks, workflow, batch runner, tests and documentation — rather than a single bulk commit.

14. Installation and Execution

14.1 Prerequisites

- Python 3.10 or later
- An OpenAI API key
- Git (for the version-control part of the project)

14.2 Setup

```
# 1. install the dependencies
pip install -r requirements.txt

# 2. copy .env.example to .env and add the key
#    OPENAI_API_KEY=sk-...

# 3. run the application
python app.py

# 4. (optional) run the tests
pytest -v
```

14.3 Cost

With gpt-4o-mini and the six sample documents, one complete run makes eighteen API calls and costs in the region of US$0.03. The MAX_CHARACTERS limit in config.py caps the size of any single request, so the cost per document cannot grow unexpectedly with an unusually long input.

15. Results

A complete run over the sample corpus processes six documents and skips one unsupported file. The console output ends with a summary table:

```
DOCUMENT              STATUS    CATEGORY          ESCALATE  CASE STATUS
PRIORITY
complaint_001.pdf     success   Billing           No        Resolved      Low
complaint_002.pdf     success   Delivery          Yes       Escalated     High
complaint_003.txt     success   Warranty          Yes       Escalated     High
complaint_004.docx    success   Account Access    Yes       Escalated     High
complaint_005.docx    success   Service Quality   No        Resolved      Low
enquiry_006.txt       success   Other             No        Closed        Low

  6 succeeded, 0 failed
```

The values above are representative of the system's behaviour on the sample corpus; exact wording, timings and token counts vary between runs. Replace this section with a screenshot of your own run before submission.

15.1 Observations

- The system correctly distinguishes the enquiry from the five complaints, setting is_complaint to No for enquiry_006 — the case that a naive keyword rule would misclassify.

- Escalation is identified from the meaning of the text, not from a keyword: complaint_005 mentions a manager but records the case as resolved, whereas complaint_002 describes a third unanswered contact and an ombudsman.
- Documents with no recorded resolution produce an email that states the case is under review, rather than inventing an outcome.
- The unsupported .csv file is skipped silently, as intended, and does not appear in the report.

16. Skills Demonstrated

Skill	Where it is demonstrated
Python	All seven files: type hints, docstrings, pathlib, comprehensions, context managers, lru_cache
LLM API integration	ai_tasks.build_llm() — ChatOpenAI configured from config.py, key read from .env
Prompt Engineering	ai_tasks.py — three prompts, system/human separation, a shared anti-hallucination rule, task-specific constraints
Structured Outputs	ai_tasks.py — with_structured_output binds each Pydantic schema to the model; no text parsing anywhere
Pydantic	models.py — three BaseModel classes, Literal for closed vocabularies, Optional for absent facts
Batch Processing	app.py — discovery of the folder, the per-document loop, the consolidated CSV
Sequential / Parallel Workflow	workflow.py — LangGraph; extract is sequential, email and summary run in parallel
Error Handling	Retry in ai_tasks.py, validation in document_reader.py, per-document try/except in app.py, start-up check in main()
Modular Code	Six modules with one responsibility each and a single direction of dependency
File Processing	document_reader.py — .txt, .pdf and .docx in; JSON, TXT and CSV out
Git/GitHub	.gitignore excluding .env and output/, a commit-by-commit history, GitHub Actions running the tests
Basic Logging	app.py — timestamped INFO/WARNING/ERROR messages to the screen and to output/run.log

17. Limitations

- Scanned documents are not supported. A PDF containing only images has no text layer, and the reader correctly rejects it rather than returning an empty extraction. Optical character recognition would be required.
- Documents are processed one at a time. The parallelism in this system is within a document, not across documents, so a batch of a thousand files would take proportionally longer.
- There is no human review step. The generated emails are written to disk, not sent; a person is assumed to review them before they reach a customer.
- Extraction quality depends on the model. A document that is genuinely ambiguous will produce an uncertain classification, and the system currently has no confidence threshold at which it would ask for human input.
- English only. The prompts and the sample corpus are in English; other languages have not been tested.

18. Future Enhancements

10. Add OCR (for example Tesseract) so that scanned complaint documents can be processed.
11. Process several documents concurrently with a thread pool, since the application spends almost all of its time waiting for the API.
12. Route escalated cases automatically to the responsible team, using the extracted category and priority.
13. Retrieve company policy documents and supply the relevant passages to the generation tasks, so the email can cite the correct refund window or service-level target.
14. Add a simple web interface so that a non-technical user can drop files in and download the report.
15. Write the structured results to a database or CRM rather than to files, enabling trend reporting across months.

19. Conclusion

The project delivers a working, end-to-end GenAI application that converts a folder of unstructured complaint documents into structured data, customer correspondence and management summaries, with no human intervention beyond starting the program.

Three design decisions carry most of the value. Binding every LLM response to a Pydantic schema removes text parsing from the system entirely and guarantees that the report is machine-readable. Expressing the three tasks as a LangGraph state machine makes the dependency between them explicit and allows the two independent tasks to run in parallel. And handling errors at three separate levels means that a network glitch, a corrupt file or a missing configuration produces a clear message and a recorded outcome rather than a lost document.

The result is a small codebase — six modules and around eight hundred lines including comments — that is straightforward to read, tested automatically on every commit, and inexpensive to run.

20. References

- OpenAI Platform Documentation — Structured Outputs. https://platform.openai.com/docs/guides/structured-outputs
- LangChain Documentation — Structured output. https://python.langchain.com/docs/how_to/structured_output/
- LangGraph Documentation — Graph API and parallel node execution. https://langchain-ai.github.io/langgraph/
- Pydantic Documentation — Models and validation. https://docs.pydantic.dev/latest/
- pypdf Documentation. https://pypdf.readthedocs.io/
- python-docx Documentation. https://python-docx.readthedocs.io/
- GitHub Actions Documentation. https://docs.github.com/en/actions

Appendix A — Project File Structure

```
complaint-processor/
│
├── app.py                      THE FILE YOU RUN — batch runner, logging, CSV
report
├── config.py                   all settings in one place
├── models.py                   three Pydantic schemas
├── document_reader.py          txt / pdf / docx  ->  plain text
├── ai_tasks.py                 three prompts, three LLM calls, retry logic
├── workflow.py                 LangGraph orchestration
├── test_basic.py               15 automated tests
│
├── data/                       input documents
│   ├── complaint_001.pdf       duplicate billing charge (resolved)
│   ├── complaint_002.pdf       delayed delivery (escalated)
│   ├── complaint_003.txt       repeat warranty failure (escalated)
│   ├── complaint_004.docx      account access / data privacy (escalated)
│   ├── complaint_005.docx      service quality (resolved)
│   ├── enquiry_006.txt         an enquiry, not a complaint
│   └── vendor_courier_notes.csv    unsupported type — skipped
│
├── output/                     generated on each run (git-ignored)
│   ├── structured_data/
│   ├── customer_emails/
│   ├── case_summaries/
│   ├── final_report.csv
│   └── run.log
│
├── .github/workflows/tests.yml    GitHub Actions
├── .env.example                template for the API key
├── .gitignore                  excludes .env and output/
├── requirements.txt
├── README.md                   how to run it
├── HOW_TO_DEMO.md              a five-minute demonstration script
├── GIT_SETUP.md                step-by-step GitHub instructions
└── SKILLS_CHECKLIST.md         the twelve skills mapped to the code
```

Appendix B — Sample Input Document

The following is complaint_003.txt from the sample corpus, shown in full. All details are fictional.

```
COMPLAINT INTAKE FORM
=====================

Case Reference: CC-2026-0455
Date Received: 21 March 2026
Channel: Phone, transcribed by agent

Customer Name: Meera Shah
Email: meera.shah@example.net
Phone: +91 99201 44580
Invoice Number: INV-2026-3391

Product/Service: NimbusHome Air Purifier X2 (purchased 14 months ago)

Complaint Description:
The unit has stopped drawing air and displays error E07 on the front panel.
This is the third time the same fault has occurred. The unit was repaired
under warranty in August 2025 and again in December 2025.

Issue Details:
The customer says the fault reappeared four days after the second repair was
returned. The device is 14 months old and still inside the 24-month hardware
warranty.

Supporting Information:
The customer emailed photographs of the error display and a copy of the
original invoice.

Resolution Provided:
A replacement unit has been requested and a loan purifier was shipped on
22 March while the replacement is prepared.

Escalation Information:
Escalated to the service manager because this is a repeat failure and the
customer has asked for confirmation in writing.

Case Status: In progress
```

Appendix C — Sample Output Format

The three artefacts produced for a single document. The content shown illustrates the format; the exact wording is generated by the model at run time.

C.1 structured_data/complaint_003.json

```
{
  "customer_name": "Meera Shah",
  "email": "meera.shah@example.net",
  "phone_number": "+91 99201 44580",
  "product_or_service": "NimbusHome Air Purifier X2",
  "complaint_category": "Warranty",
  "issue_description": "The unit has stopped drawing air and displays error
E07. ...",
  "resolution_provided": "A replacement unit has been requested and a
loan ...",
  "is_complaint": true,
  "escalation_required": true,
  "supporting_document_available": true,
  "case_status": "In Progress",
  "priority": "High"
}
```

C.2 customer_emails/complaint_003.txt

```
Subject: Your NimbusHome Air Purifier X2 — replacement in progress

Dear Meera Shah,

Thank you for contacting us about your NimbusHome Air Purifier X2. We are
sorry that the same fault has now occurred for a third time.

Our records show that the unit stopped drawing air and displayed error E07,
and that this follows repairs carried out in August and December 2025.

A replacement unit has been requested, and a loan purifier was shipped to
you on 22 March so that you are not left without a device. Your case has
also been escalated to our service manager.

If there is anything further you would like to add, please reply to this
email and your response will be added to the existing case.

Kind regards,
Customer Care Team
```

C.3 case_summaries/complaint_003.txt

```
INTERNAL CASE SUMMARY
=====================

Case overview          : Third recurrence of fault E07 on a 14-month-old
                         air purifier, still within the 24-month warranty.

Key issue              : The same fault has returned after two warranty
                         repairs, four days after the second return.
```

```
Action taken           : Replacement requested; loan unit shipped 22 March.

Current status         : In Progress — escalated to the service manager.

Recommended next action: Service manager to confirm the replacement date to
                         the customer in writing within 24 hours.
```